Queue Wait Category Predictor

SI Lab Project | B.Tech CSE (AI & DS), Semester V | Ramdeobaba University

Name: Harshita Sachdev Roll No: 06 Section/Batch: B1-B1

Input: token number, time slot, day **Output:** wait category (Short / Medium / Long) and current position in the queue

Run all cells from top to bottom. Headings match the report sections.

0. Install and import libraries

```
!pip install -q gradio

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from datetime import datetime
from zoneinfo import ZoneInfo

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, cl
```

1. Dataset Generation and Saving (CSV)

A real queue log is not available, so a realistic queue is simulated. For every day and time slot the tokens start from 1. Each student takes 3 to 8 minutes to be served (busy slots and Mondays/Fridays are slower). Waiting time is the total service time of all students ahead. It is then converted into a category:

Category	Waiting time
Short	up to 15 min
Medium	16 to 40 min
Long	more than 40 min

```
np.random.seed(42)
days  = ["Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"]
slots = ["09:00-10:00","10:00-11:00","11:00-12:00","12:00-13:00","14:00-15:00","15:00-16:00"]
slot_factor = {"11:00-12:00":1.3, "12:00-13:00":1.2}
day_factor  = {"Monday":1.2, "Friday":1.1}

rows = []
for week in range(8):
    for d in days:
        for s in slots:
            n = np.random.randint(8, 31)
            f = slot_factor.get(s,1.0) * day_factor.get(d,1.0)
            service = np.random.uniform(3, 8, n) * f
            for i in range(n):
                rows.append({"day": d, "time_slot": s, "token_number": i+1,
                             "waiting_time_min": round(service[:i].sum(), 1)})
df = pd.DataFrame(rows)

def categorize(w):
    if w <= 15: return "Short"
    elif w <= 40: return "Medium"
    return "Long"
df["wait_category"] = df["waiting_time_min"].apply(categorize)
```

```
df.to_csv("queue_dataset.csv", index=False)
print("Dataset saved: queue_dataset.csv")
```

```
Dataset saved: queue_dataset.csv
```

2. Dataset Loading and Exploration

```
df = pd.read_csv("queue_dataset.csv")
print("Shape (rows, columns):", df.shape)
print("\nColumns and data types:")
print(df.dtypes)
print("\nFirst 5 rows:")
display(df.head())
```

```
Shape (rows, columns): (5345, 5)

Columns and data types:
day                 object
time_slot           object
token_number         int64
waiting_time_min   float64
wait_category       object
dtype: object

First 5 rows:
```

	day	time_slot	token_number	waiting_time_min	wait_category
0	Monday	09:00-10:00	1	0.0	Short
1	Monday	09:00-10:00	2	8.4	Short
2	Monday	09:00-10:00	3	13.1	Short
3	Monday	09:00-10:00	4	21.4	Medium
4	Monday	09:00-10:00	5	28.5	Medium

```
print("Missing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
print("\nClass distribution:")
print(df["wait_category"].value_counts())
```

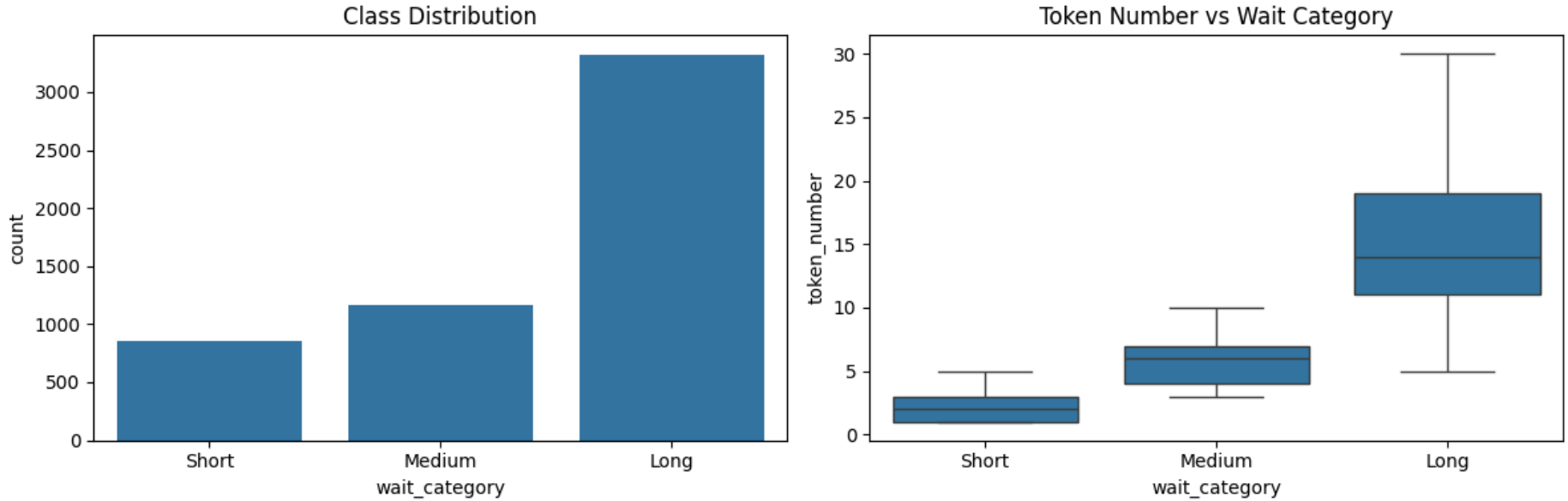
```
Missing values:
 day                 0
time_slot           0
token_number        0
waiting_time_min    0
wait_category       0
dtype: int64

Duplicate rows: 354

Class distribution:
```

```
wait_category
Long      3322
Medium    1167
Short      856
Name: count, dtype: int64
```

```
fig, ax = plt.subplots(1, 2, figsize=(12,4))
sns.countplot(x="wait_category", data=df, order=["Short","Medium","Long"], ax=ax[0])
ax[0].set_title("Class Distribution")
sns.boxplot(x="wait_category", y="token_number", data=df, order=["Short","Medium","Long"], ax=ax[1])
ax[1].set_title("Token Number vs Wait Category")
plt.tight_layout(); plt.show()
```



3. Data Preprocessing

Remove duplicate rows, encode day and time slot (one-hot), scale the token number, separate features and target, and split into train and test sets.

```
before = len(df)
df = df.drop_duplicates().reset_index(drop=True)   # e.g. token 1 repeats with 0 min wait
print("Duplicates removed:", before - len(df), "| Rows now:", len(df))

X = df[["day", "time_slot", "token_number"]]
y = df["wait_category"]

preprocess = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["day", "time_slot"]),
    ("num", StandardScaler(), ["token_number"])
])

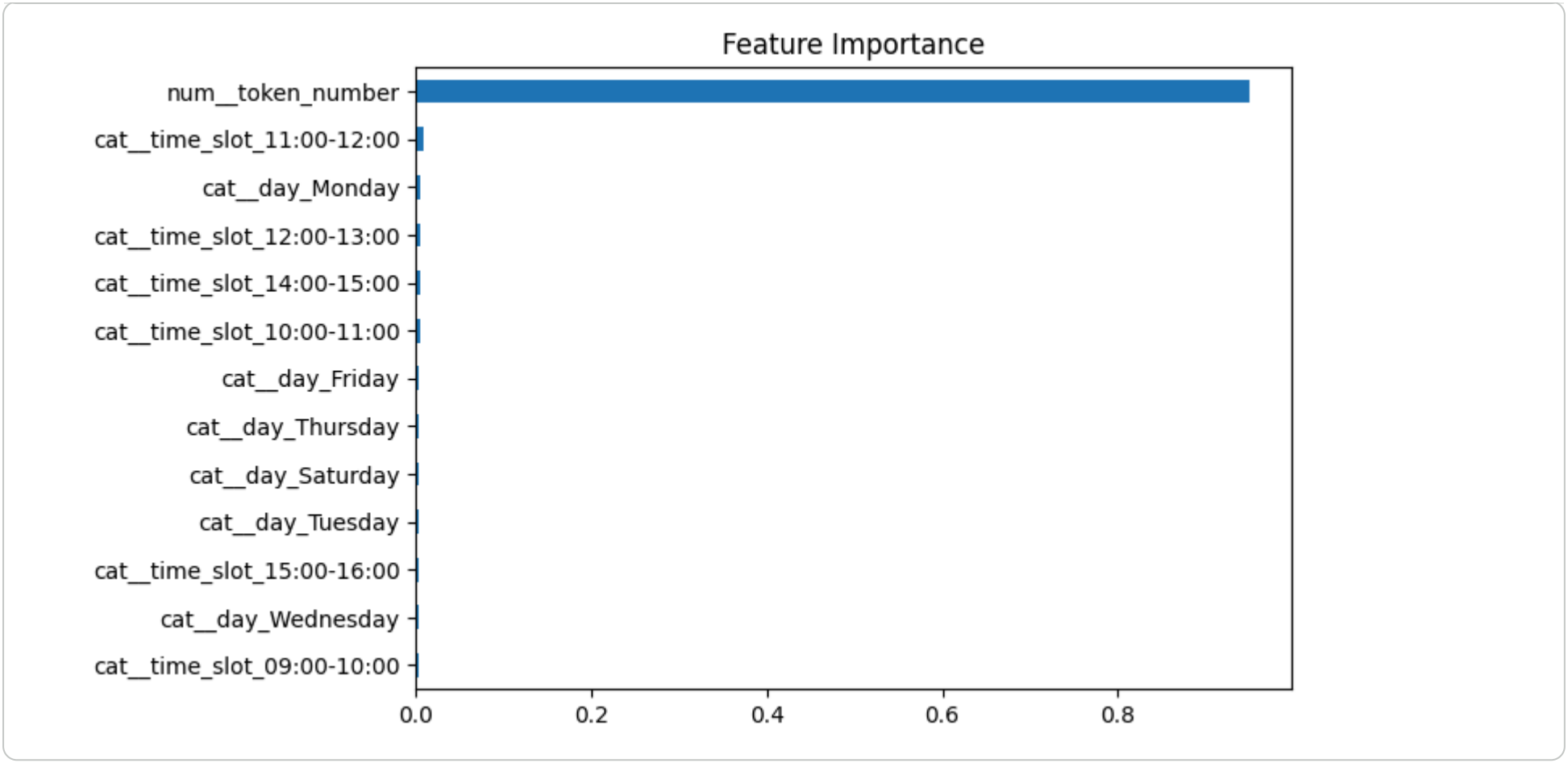
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
```

```
Duplicates removed: 354 | Rows now: 4991
Training samples: 3992
Testing samples : 999
```

4. Feature Selection

All three inputs (day, time slot, token number) are used because they are exactly what a student provides. No extra feature extraction is required. The plot below shows which inputs matter most to the Random Forest.

```
rf_check = Pipeline([("prep", preprocess), ("m", RandomForestClassifier(random_state=42))])
rf_check.fit(X_train, y_train)
names = rf_check.named_steps["prep"].get_feature_names_out()
imp = pd.Series(rf_check.named_steps["m"].feature_importances_, index=names).sort_values()
imp.plot(kind="barh", figsize=(7,5), title="Feature Importance")
plt.show()
```



5. Training Multiple Classification Models

1. Logistic Regression: simple baseline
2. Decision Tree: rule-based, easy to explain
3. Random Forest: many trees, usually more accurate
4. K-Nearest Neighbors: classifies by similar past tokens

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(max_depth=8, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

trained, results = {}, []
for name, m in models.items():
    pipe = Pipeline([("prep", preprocess), ("model", m)])
    pipe.fit(X_train, y_train)
    trained[name] = pipe
    pred = pipe.predict(X_test)
    results.append({
        "Model": name,
        "Accuracy":  round(accuracy_score(y_test, pred), 4),
        "Precision": round(precision_score(y_test, pred, average="weighted", zero_division=0), 4),
        "Recall":    round(recall_score(y_test, pred, average="weighted"), 4),
        "F1-Score":  round(f1_score(y_test, pred, average="weighted"), 4)
    })
print("All 4 models trained.")
```

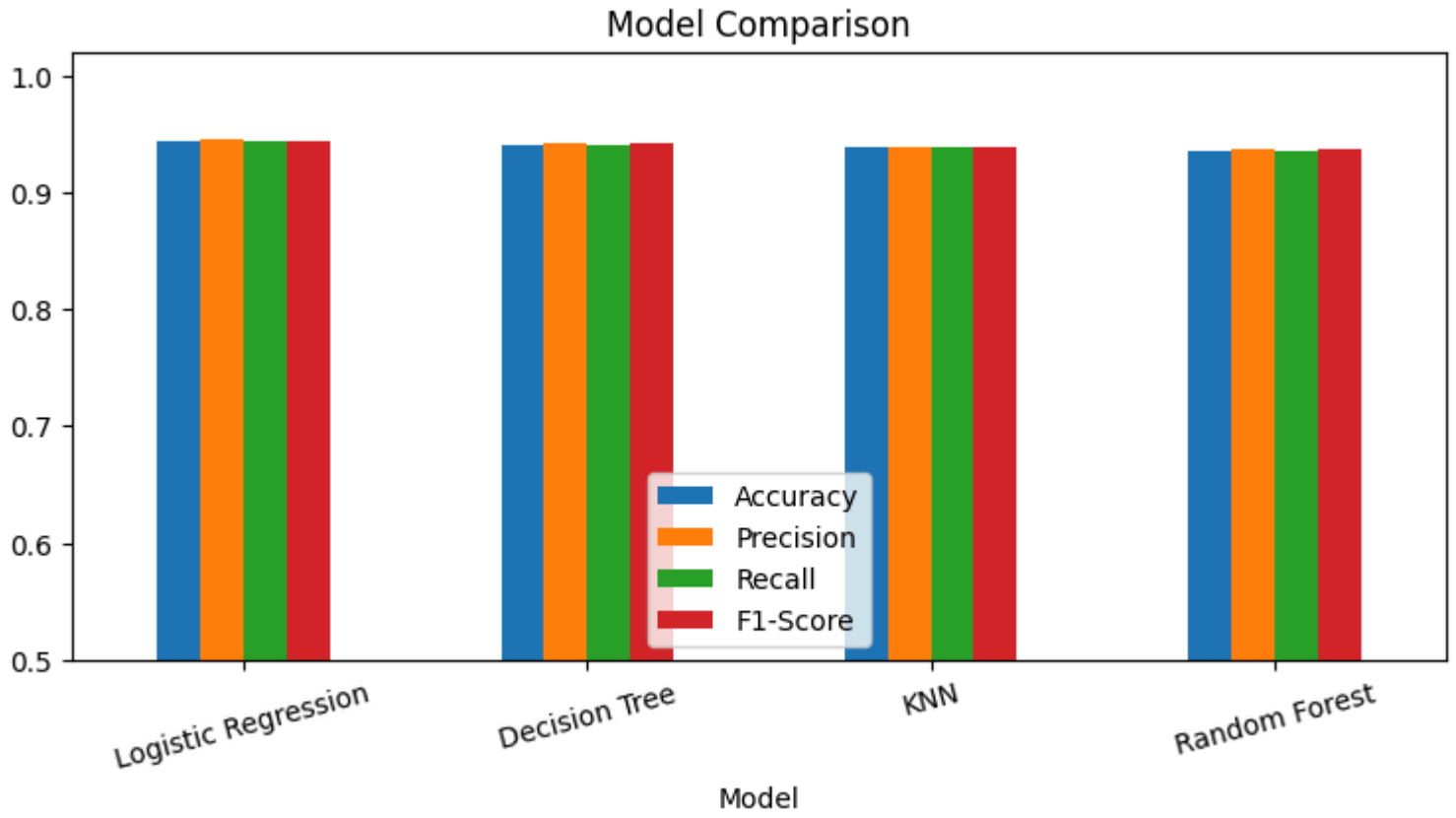
```
All 4 models trained.
```

6. Performance Evaluation and Model Comparison

```
results_df = pd.DataFrame(results).sort_values("F1-Score", ascending=False).reset_index(drop=True)
display(results_df)

results_df.set_index("Model")[["Accuracy","Precision","Recall","F1-Score"]].plot(
    kind="bar", figsize=(9,4), ylim=(0.5,1.02), title="Model Comparison")
plt.xticks(rotation=15); plt.show()
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.9439	0.9445	0.9439	0.9442
1	Decision Tree	0.9409	0.9419	0.9409	0.9413
2	KNN	0.9379	0.9380	0.9379	0.9378
3	Random Forest	0.9359	0.9363	0.9359	0.9361

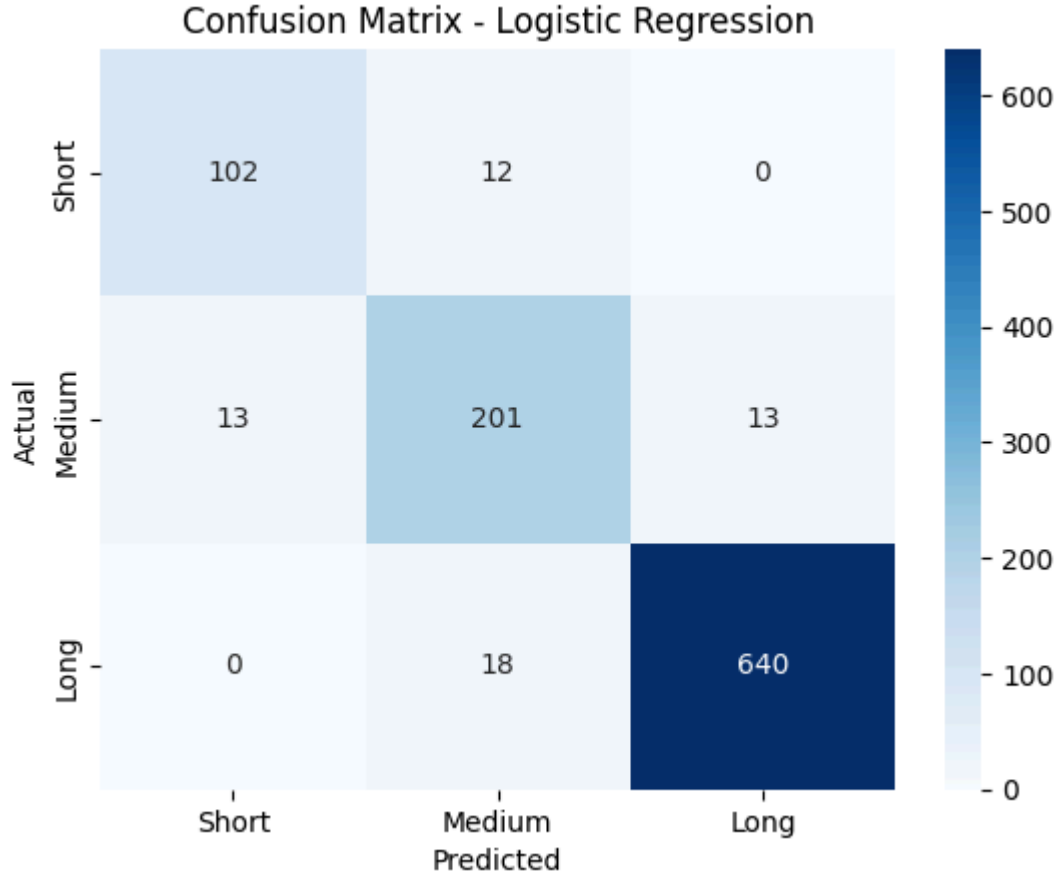


7. Best Model Selection and Confusion Matrix

```
best_name = results_df.iloc[0]["Model"]
best_model = trained[best_name]
print("Best model:", best_name)

pred = best_model.predict(X_test)
labels = ["Short","Medium","Long"]
cm = confusion_matrix(y_test, pred, labels=labels)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title(f"Confusion Matrix - {best_name}")
plt.show()
print(classification_report(y_test, pred))
```

```
Best model: Logistic Regression
```



	precision	recall	f1-score	support
Long	0.98	0.97	0.98	658
Medium	0.87	0.89	0.88	227
Short	0.89	0.89	0.89	114
accuracy			0.94	999
macro avg	0.91	0.92	0.91	999
weighted avg	0.94	0.94	0.94	999

8. Save and Load the Best Model

```
joblib.dump(best_model, "best_queue_model.pkl")
loaded_model = joblib.load("best_queue_model.pkl")
print("Model saved as best_queue_model.pkl and loaded again successfully.")
```

```
Model saved as best_queue_model.pkl and loaded again successfully.
```

9. Prediction Function

Position in queue = token number minus the students already served. If the chosen day and slot is happening **right now** (India time), the number served is estimated from the elapsed time (about 5 minutes per student). Otherwise the queue is assumed to start from the beginning of the slot.

```
AVG_SERVICE = 5  # minutes per student

tips = {"Short":  "You will be served soon.",
        "Medium": "Come back in a while, you have some time.",
        "Long":   "Long wait expected. Consider coming later."}

def predict_queue(token_number, time_slot, day):
    token_number = int(token_number)
    row = pd.DataFrame([{"day": day, "time_slot": time_slot, "token_number": token_number}])
    category = loaded_model.predict(row)[0]

    now = datetime.now(ZoneInfo("Asia/Kolkata"))
    start = datetime.strptime(time_slot.split("–")[0], "%H:%M").time()
    end   = datetime.strptime(time_slot.split("–")[1], "%H:%M").time()
    served = 0
    if now.strftime("%A") == day and start <= now.time() <= end:
        elapsed = (now.hour*60 + now.minute) - (start.hour*60 + start.minute)
        served = int(elapsed // AVG_SERVICE)
    position = max(1, token_number - served)
```

```
    return category, position, tips[category]

print(predict_queue(12, "11:00-12:00", "Monday"))
```

```
('Long', 12, 'Long wait expected. Consider coming later.')
```

10. GUI using Gradio

The student enters token number, time slot and day, and gets the wait category and queue position. Run the cell and use the link or the form shown below it.

```
import gradio as gr

def gui_predict(token_number, time_slot, day):
    category, position, tip = predict_queue(token_number, time_slot, day)
    return category, position, tip

app = gr.Interface(
    fn=gui_predict,
    inputs=[
        gr.Number(label="Token Number", value=10, precision=0, minimum=1),
        gr.Dropdown(slots, label="Time Slot", value="11:00-12:00"),
        gr.Dropdown(days, label="Day", value="Monday")
    ],
    outputs=[
        gr.Textbox(label="Wait Category"),
        gr.Number(label="Position in Queue Right Now"),
        gr.Textbox(label="Advice")
    ],
    title="Queue Wait Category Predictor",
    description="Enter your token number, time slot and day to know your wait category and queue position
)
app.launch(share=True)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://d856fee4faa28cb890.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU
```

